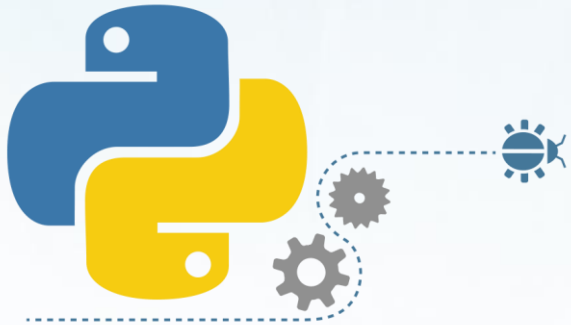
TRƯỜNG ĐẠI HỌC
SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH
HCMC University of Technology and Education

KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN HỆ THỐNG THÔNG TIN

NHẬP MÔN LẬP TRÌNH PYTHON (IPPA233277)

CẤU TRÚC LẶP



1. Hiểu và vận hành được cấu trúc vòng lặp: while, for, while/else, for/else
2. Hiểu và vận hành được biểu thức break, continue
3. Sử dụng được cấu trúc lồng của vòng lặp

1. Vòng lặp while
2. Vòng lặp for
3. Câu lệnh break
4. Câu lệnh continue
5. Câu lệnh while-else
6. Câu lệnh for-else
7. Vòng lặp lồng nhau

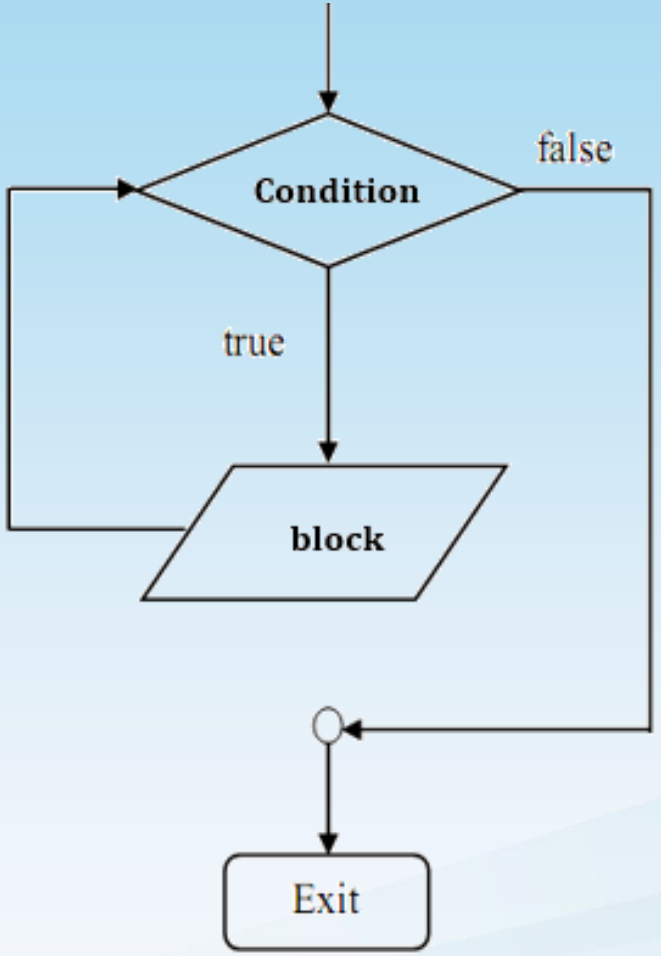
- Cấu trúc lặp là một trong những cấu trúc quan trọng trong lập trình, hỗ trợ việc lặp đi lặp lại một câu lệnh hoặc hành động một cách tự động và dễ dàng.

- while dùng để yêu cầu một công việc được lặp đi lặp lại. Nếu condition là True thì block sẽ được lặp lại

Cú pháp:

```
while condition :
    block
```

- Có thể block sẽ không được thực hiện lần nào nếu condition là False từ đầu
- Kết thúc vòng lặp while bằng các đưa condition về False hoặc dùng break



Ví dụ: Viết chương trình yêu cầu nhập vào một số nguyên dương [1..10], nếu nhập sai yêu cầu nhập lại. Khi nhập đúng thì xuất ra bình phương của giá trị mới nhập vào.

```
n = -1
while n < 1 or n > 10:
    n = int(input('Nhập giá trị [1..10]: '))
print('result = ', pow(value, 2))
```

Ví dụ: Viết chương trình yêu cầu nhập vào một số nguyên dương (1—10), nếu nhập sai yêu cầu nhập lại. Khi nhập đúng thì xuất ra tổng các số từ 1 ➔ N.

```
n = -1; s = 0; i = 1
while n < 1 or n > 10:
    n = int(input('Nhập giá trị [1..10]: '))
while i <= n:
    s += i
    i += 1
print('Tổng =', s)
```

- Dùng để lặp tuần tự các công việc, hoặc duyệt qua các phần tử trong list, set, string,…
- Phần thân vòng lặp for được bắt đầu bằng thụt đầu dòng

Cú pháp:

```
for val in sequence:
    # loop body
```

Trong đó: val là giá trị của từng phần tử trong sequence (list, set, string, …) được duyệt qua trong mỗi lần lặp. Vòng lặp sẽ tiếp tục cho đến phần tử cuối cùng trong sequence.

Ví dụ:

```
# List of numbers
numbers = [6, 5, 3, 8, 4]
for i in numbers:
    print(i)
```

```
# String
s = "Hello World"
print("String Iteration")
for i in s:
    print(i)
```

- Ngoài ra, có thể sử dụng range để định nghĩa vùng dữ liệu lặp và bước lặp

Cú pháp hàm range:

range (begin, end [, step])

Trong đó:

begin : giá trị bắt đầu (default: 0)

end : giá trị cuối

step : bước nhảy (default: 1)

Ví dụ:

- range(10) → 0; 1; 2; 3; 4; 5; 6; 7; 8; 9
- range(1, 10) → 1; 2; 3; 4; 5; 6; 7; 8; 9
- range(1, 10, 2) → 1; 3; 5; 7; 9
- range(10, 0, -1) → 10; 9; 8; 7; 6; 5; 4; 3; 2; 1
- range(10, 0, -2) → 10; 8; 6; 4; 2
- range(2, 11, 2) → 2; 4; 6; 8; 10

```
for i in range(7): # start=0, stop=7, step=1
    print(i, end=' ')
```

```
0 1 2 3 4 5 6
```

```
for j in range(1, 7): # start=1, stop=7, step=1
    print(j, end=' ')
```

```
1 2 3 4 5 6
```

```
for k in range(1, 7, 2): # start=1, stop=7, step=2
    print(k, end=' ')
```

```
1 3 5
```




- Hàm range() cũng đại diện cho chỉ mục (index) của một list để duyệt các phần tử trong list

Ví dụ:

```
job = ['teacher', 'singer', 'football player']
# iterate over the list using index
for i in range(len(job)):
    print("I want to be a", job[i])
```

```
I want to be a teacher
I want to be a singer
I want to be a football player
```

- Từ khóa break dùng để thoát khỏi vòng lặp khi thỏa mãn yêu cầu.
- Gặp lệnh break, chương trình sẽ không thực hiện bất kỳ lệnh nào bên dưới mà thoát luôn khỏi vòng lặp
- Trong trường hợp vòng lặp a chứa vòng lặp b. Trong vòng lặp b chạy câu lệnh break thì chỉ vòng lặp b kết thúc, còn vòng lặp a thì không

Ví dụ: Viết chương trình vòng lặp cho phép phần mềm chạy liên tục, khi nào hỏi thoát mới thoát phần mềm

```
while True:
    a = int(input('Nhập giá trị: '))
    print('Giá trị bạn nhập', a)
    s = input('Tiếp tục thực hiện? (c/k): ')
    if s == “k":
        break
print('Tạm biệt!')
```

- Từ khóa continue dùng để nhảy sớm tới lần lặp kế tiếp, các câu lệnh dưới continue sẽ không được thực thi

```
while expression:
    # while-block-1
    continue
    # while-block-2
```

Sau khi thực hiện xong while-block-1, câu lệnh continue sẽ tiếp tục vòng lặp mà không quan tâm những câu lên bên dưới, tức sẽ bỏ qua while-block-2

Ví dụ: Tính tổng các chữ số lẻ từ 1 → 15, ngoại trừ số 3 và số 11

```
sum = 0
for i in range(1, 16, 2):
    if i in (3, 11):
        continue
    sum += i
print(sum)
```

➔ 50

- Python hỗ trợ khối else trong trường hợp while kết thúc một các bình thường (tức không dùng break để kết thúc)

```
while expression:
    # while-block
else:
    # else-block
```

Nếu while kết thúc một cách bình thường thì else-block sẽ tự động được thực hiện ngay sau đó

```
k = 0
while k < 3:
    print("Value of k = ", k)
    k += 1
else:
    print("k now >= 3")
```

```
Value of k =  0
Value of k =  1
Value of k =  2
k now >= 3
```

Ví dụ:

```
count = sum = 0
print('Nhập danh sách các số dương để tính trung bình: ')
while count < 5:
    val = float(input('Nhập số: '))
    if val < 0:
        print('Số 0 sai quy tắc, thoát phần mềm!')
        break
    count += 1; sum += val
else:
    print('Trung bình =', sum/count)
```

Nếu nhập số < 0 thì sẽ thoát while và bên trong else cũng không được thực thi (do kết thúc bằng lệnh break). Nếu nhập đúng toàn bộ giá trị, sau khi while chạy xong thì else sẽ tự động được gọi (kết thúc while bình thường)



- Python hỗ trợ else block trong trường hợp for kết thúc một cách bình thường (tức là không phải dùng break để kết thúc). Nếu for kết thúc một cách bình thường thì else-block sẽ tự động được thực hiện ngay sau đó.

```
a = int(input('Nhập a: '))
sum = 0
for i in range(5, 10):
    if 4 % a is 1:
        print('Ngừng vòng lặp!')
        break
    sum += i
else:
    print('Tổng = ', sum)
```

Cú pháp:

```
for expression:
    # for-block
else:
    # else-block
```




- Python cũng như các ngôn ngữ khác, có thể viết các vòng lặp lồng nhau.

Ví dụ:

```
n = int(input('Nhập chiều cao: '))
for i in range(n):
    for j in range(n):
        if j == 0 or i == j or j == n - 1:
            print('*', end = ' ')
        else:
            print(' ', end = ' ')
    print()
```

